

App Dev 1 Project Report

1. Student Details

Name: Polagangu Pranavi

Roll Number: BS23f2004509

Email: 23f2004509@ds.study.iitm.ac.in

About Me: I am a student at IIT Madras BS Degree program with interest in application development. I like building applications that combine learning and user experience.

2. Project Details

Project Title: Placement Portal

Problem Statement:

Required to build a Placement Portal Application web application that allows Admin (Institute), Company, and students to interact with the system based on their roles. Institutes need efficient systems to manage campus recruitment activities involving companies, students, and placement drives.

Approach:

The app was built using Flask as the backend framework with a modular structure. It allows admin to manage company, students and drive data, while students can register and apply for company drives, and companies can create drives and manage student registrations with approval from admin.

3. AI/LLM Declaration

I used **Gemini** to assist in writing database model definitions, to visualise routing, and to improve variable and methods naming consistency.

The extent of AI/LLM usage is around **15–20%**, limited to **code suggestions and documentation formatting**.

All final implementation logic, debugging, and integration were done manually.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework
SQLAlchemy	Object Relational Mapper for SQLite database
Jinja2	Template engine for rendering dynamic HTML pages
Bootstrap 5	Frontend styling and responsive design
Werkzeug Security	Password hashing and verification
SQLite	Lightweight local database for storing user data

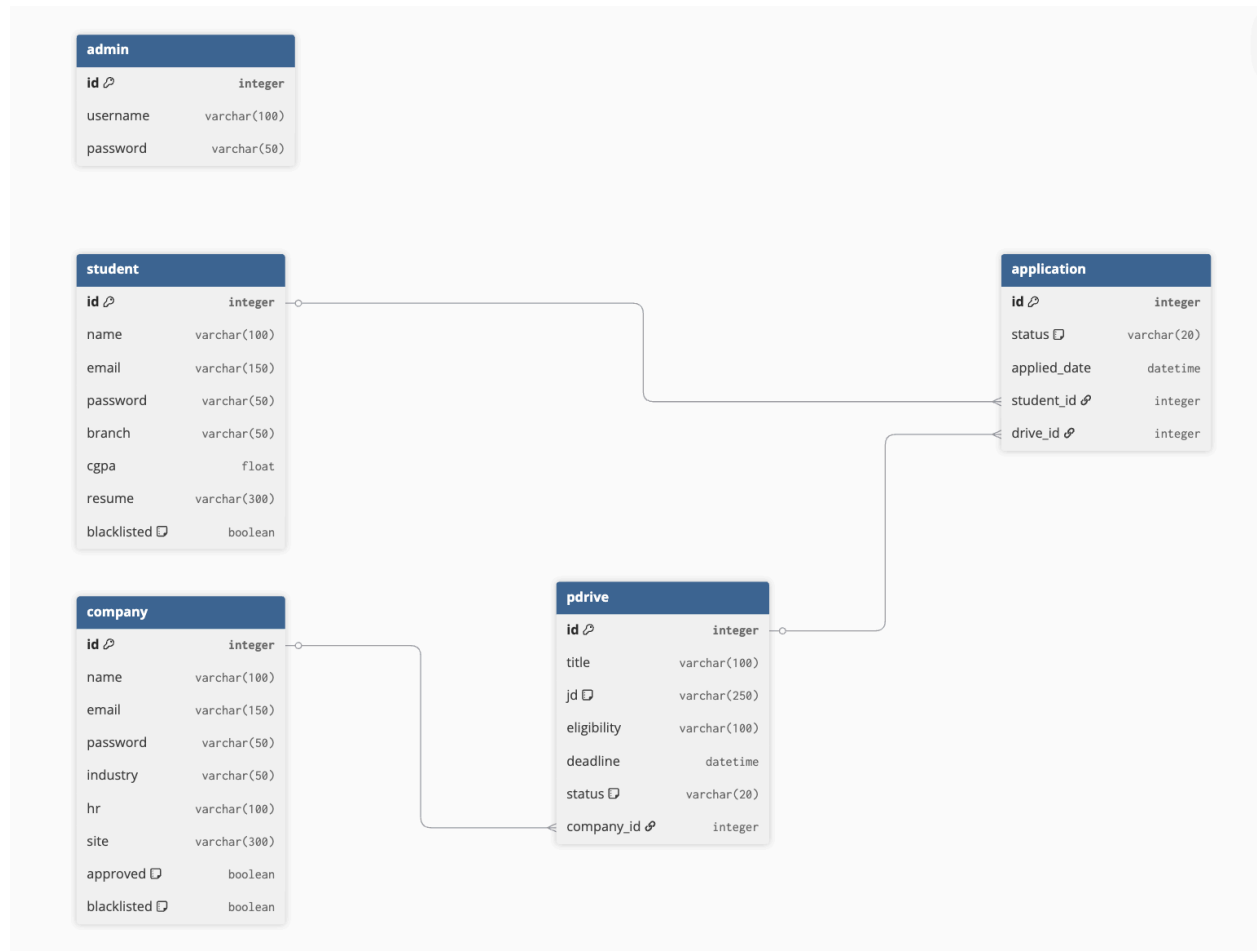
5. Database Schema / ER Diagram

Tables:

1. **Admin** : Stores administrator login credentials (id, username, password)
2. **Student** : Stores student profile and academic details (id, name, email, password, branch, cgpa, resume, blacklisted)
3. **Company** : Stores company profile details (id, name, email, password, industry, hr, site, approved, blacklisted)
4. **Drive** : Stores job/internship opportunities posted by companies (id, title, jd, eligibility, deadline, status, company_id)
5. **Application** : Logs the job applications submitted by students (id, status, applied_date, student_id, drive_id)

Relationships:

- One-to-Many → Company → Drive (A single company can host multiple placement drives)
- One-to-Many → Student → Application (A single student can submit multiple job applications)
- One-to-Many → Drive → Application (A single placement drive can receive applications from multiple students)



6. API Resource Endpoints

Endpoint	Method	Description
/login	POST	Authenticate user (Admin, Company, or Student) and generate session
/register/student	POST	Register a new student account into the system
/register/company	POST	Register a new company account (requires Admin approval to become active)

/company/drive/create	POST	Add a new placement drive entry (Company only)
/student/apply/<drive_id>	POST	Submit an application for a specific placement drive (Student only)
/company/application/<app_id>/status	POST	Update the status of a specific student application (e.g., Shortlisted, Selected)
/admin/companies/approve/<company_id>	GET	Approve a registered company's profile (Admin only)
/admin/drives/approve/<drive_id>	GET	Approve a newly posted placement drive (Admin only)

YAML API Definition File:

Included separately in the submission ZIP as [api.yaml](#).

7. Architecture and Features

Architecture Overview:

- **app.py** : Main Flask application entry point containing all route controllers and core logic
- **model.py** : Database models and schema configurations using SQLAlchemy
- **/templates** : Jinja2 HTML templates organized meticulously by user roles (/admin, /company, /student)
- **placements.db** : SQLite local database file (automatically generated) holding application data

Implemented Features:

- Role-Based Access Control (RBAC): Distinct registration, login, and customized dashboard environments for three user types: Admins, Companies, and Students.
- Administrative Moderation: Admin tools to review/approve newly registered companies and newly posted placement drives before they go live, ensuring platform quality.
- Drive & Application Management: Companies can create placement workflows (JD, eligibility, deadlines), and students can browse approved drives and submit their applications.

- Application Tracking Pipeline: Companies can update application statuses progressively (Applied -> Shortlisted -> Selected -> Rejected) for transparent tracking.
 - Blacklisting System: Admins can blacklist problematic companies or students to restrict their capabilities on the portal.
 - Secure Authentication: Secure password hashing (via Werkzeug) and server-side route protection checking user session roles.
 - Responsive UI: Frontend interactions structured with Bootstrap 5 components and Jinja2 template inheritance (base.html).
-

8. Video Presentation

Drive Link:

https://drive.google.com/file/d/11qJ1X2_paMoBjgER1a0AESiL8Mw-wURS/view?usp=sharing